Hardware-Aware Autotuning of a CUDA GEMM Kernel with Response Surface Methodology

Vaidheesh Bharadwaj
Department of Mathematics
Manipal Institute of Technology
Manipal Academy of Higher Education
Manipal, India
Reg. No. 230957088

Prof. Sunita Sharma (Guide)
Department of Mathematics
Manipal Institute of Technology
Manipal Academy of Higher Education
Manipal, India

***Abstract*—A single parameterised CUDA matrix-multiplication kernel was tuned over six compile-time factors on an NVIDIA H200, using a 2^{6-1} fractional-factorial screen, a 6-factor central composite design, a second-order response surface, constrained sequential quadratic programming and measured validation. Every result is checked against an exhaustive sweep of all 1,396 feasible configurations. The pipeline selected the second-best configuration of 1,396 (0.17% from the optimum) after measuring 59. Random search with the same budget did as well in 7.8% of 10,000 trials and needs a median of 424 evaluations to match it. The pre-registered second-order surface was statistically inadequate (lack of fit 7.8% against a 5% limit), because of 3-factor interactions and abrupt compiler effects, so the final choice was made by measurement. The selected kernel reaches 72.0% of cuBLAS FP32 throughput and is 2.6× faster than the untuned default.**

***Index Terms*—Autotuning, central composite design, CUDA, design of experiments, fractional factorial design, GEMM, GPU performance, response surface methodology.**

I. Introduction

The same kernel source runs at 13.2 TFLOPS with textbook tile sizes and 34.3 TFLOPS with tuned ones: six compile-time numbers decide a 2.6× difference. Choosing those numbers is usually done by exhaustive search, which grows with every new factor, or by machine-learning autotuners such as TVM [1] and Ansor [2], which need many samples. This project tests whether classical Design of Experiments (DOE) and Response Surface Methodology (RSM) [3], [4] can find a near-best configuration from a small, statistically designed sample.

A. Problem Statement

Given a parameterised CUDA SGEMM kernel with six compile-time factors and 1,396 feasible configurations on an NVIDIA H200, find a configuration close to the fastest one while measuring only a small fraction of the space, and quantify how close it gets compared with exhaustive search and with random search on the same budget.

B. Objectives

The study answers five research questions. RQ3 is the primary claim; RQ5 is context.

1. *RQ1:* Can a fractional factorial design separate the parameters that matter from noise?
2. *RQ2:* Does a second-order response surface capture the GPU's non-linear behaviour?
3. *RQ3:* With the same number of trials, does DOE–RSM beat random search?
4. *RQ4:* How close does DOE–RSM get to the exhaustive-search optimum?
5. *RQ5:* What share of cuBLAS throughput does the tuned kernel reach?

II. Related Work

Empirical autotuners measure candidate configurations on the target hardware and use a search strategy to decide what to measure next. OpenTuner [5] is a general framework that runs an ensemble of search techniques and gives more trials to whichever is making progress. CLTune [6] and Kernel Tuner [7] target GPU kernels specifically and offer strategies such as random search, simulated annealing, genetic algorithms and particle swarm optimisation. These tools find fast configurations, but their output is a configuration, not an account of which parameters matter or how they interact.

Compiler-based autotuners for deep learning replace part of the measurement with a learned cost model. TVM [1] trains a model on measured schedules and uses it to rank unmeasured ones, and Ansor [2] adds automatic construction of the search space and evolutionary search guided by such a model. Bayesian optimisation [8] is the closest relative of this project: a Gaussian-process surrogate is fitted to the measurements so far and chooses the next configuration, and it compared favourably with the other strategies in Kernel Tuner. All of these evaluate configurations one after another, and their models are built to guide the search rather than to be interpreted.

On the kernel side, Volkov and Demmel [9] showed by hand-tuning dense linear algebra on GPUs that blocking the computation in registers is central to GEMM performance; the register-tile factors TM and TN used here follow from that observation.

Response surface methodology began with Box and Wilson [3] as a sequence of designed experiments and fitted low-order polynomials for locating optimum operating conditions, and factorial, fractional factorial and central composite designs are now standard tools [4], [10]. RSM is widely used to optimise chemical and manufacturing processes, but to the author's knowledge it has not been evaluated for GPU kernel tuning against an exhaustive ground truth. This project makes that comparison, and reports where the second-order model is inadequate as well as where the design succeeds.

III. Methodology and Implementation

One parameterised kernel was tuned over six factors, with every rule of the experiment fixed in a written contract before any final data was collected.

A. Kernel and Design Space

The kernel is a 2D block-tiled shared-memory SGEMM [11], [12] computing $C = A \cdot B$ (row-major, FP32 inputs and accumulation, CUDA-core FFMA only, no Tensor Cores). The six factors (Table I) are injected as compile-time macros.

TABLE I
TUNABLE FACTORS AND THEIR LEVELS

FACTOR	MEANING	LEVELS
BM	block output-tile rows (shared memory)	32, 64, 128, 256
BN	block output-tile columns (shared memory)	32, 64, 128, 256
BK	K-depth of each shared-memory tile	8, 16, 32, 64
TM	output rows per thread (register tile)	2, 4, 8
TN	output columns per thread (register tile)	2, 4, 8
UNROLL	unroll factor of the inner dot-product loop	1, 2, 4, 8

Of the 2,304 combinations, 1,396 are feasible. The rest break a hardware or tiling constraint: more than 1,024 threads per block (528), shared memory over 48 KiB (168), a tile-load overflow (140) or a fractional load stride (72). The feasibility filter was checked against the compiler on all 2,304 candidates with no disagreements, and the kernel carries the same rules as compile-time assertions.

B. Platform

NVIDIA H200 (sm_90, 132 SMs), driver 595.71.05, CUDA 13.0.88, cuBLAS 13.1.1.3, `nvcc -O3`. The GPU is shared with other users' processes.

C. Measurement Protocol

Timing: Each launch is timed individually with a CUDA event pair; allocation, transfers, verification and compilation are outside the timed region. A measurement block is the median of 30 launches after 10 warm-up launches at 4096^3 (80 and 25 at 2048^3; 300 and 100 at 1024^3).

Clock: The SM clock was locked for the whole run; the driver held 1830 MHz (1980 MHz was requested) on all 2,001 measurements. A stability gate measured two configurations in 10 separated blocks: coefficient of variation 0.033% and 0.106%, against a 1% limit.

Randomisation: Every experiment ran as replicate blocks, each a fresh random permutation of the design, so replicates of one point never ran back to back.

Shared GPU: NVML per-process accounting checked every measurement window; a window in which another process used the SMs was repeated, up to six attempts.

Correctness: Every configuration was checked against cuBLAS once per shape by relative Frobenius error, with a tolerance of 10^{-5} fixed before any data. A deliberately broken kernel produced an error of 0.044, matching the predicted $\sqrt{(8/4096)}$, and both the custom kernel and cuBLAS agree with a float64 CPU reference to within 6×10^{-7}.

D. Screening Design

A 2^{6-1} half fraction [4] with generator F = ABCDE (defining relation I = ABCDEF, resolution VI): main effects are aliased only with 5-factor interactions and 2-factor interactions only with 4-factor interactions. Each factor ran at two legal levels (BM, BN 64/128; BK 8/32; TM, TN 4/8; UNROLL 1/8). There are no centre points, because the midpoint of a power-of-two pair is not a legal configuration; pure error comes from 3 replicate blocks (96 measurements per shape). The screen was run at 4096^3, 2048^3 and 1024^3. The pre-registered retention rule keeps a factor if its main effect or any interaction containing it has $p < 0.05$.

E. Central Composite Design and Response Surface

A 6-factor central composite design (CCD) [3], [10] in $\log_2$-coded units: the 32-run screening design as its factorial core, 12 axial points at the rotatable $\alpha = 2.378$ and 4 centre points. Each coded point was mapped to the nearest legal power of two (ties to the lower level) and the model was fitted on the realised coded values, so the centre runs as (64, 64, 16, 4, 4, 2). In total, 48 points × 3 randomised blocks = 144 measurements on 45 distinct configurations.

Second-order models were fitted by ordinary least squares with a block term, so the full quadratic has 28 polynomial terms plus 2 block effects (30 parameters). The pre-registered adequacy gate requires the lack-of-fit mean square to be at most 5% of the model mean square. Held-out error leaves out one configuration (all its replicates) at a time. Residuals were checked with the Durbin–Watson [13], Breusch–Pagan [14] and Shapiro–Wilk [15] tests and Cook's distance [16].

F. Optimisation and Validation

Optimisation was limited to the region the factorial core tested jointly (every coded factor within $[-1, 1]$), which leaves 192 feasible lattice configurations. Two methods proposed candidates: sequential quadratic programming [17] (SciPy SLSQP [18], 30 random starts) on the continuous relaxation, with the thread-count, shared-memory and tile-load limits as nonlinear constraints, then projected to the lattice with its feasible ±1-level neighbourhood; and exhaustive evaluation of each fitted model over the 192 lattice configurations. The candidates were measured in 5 randomised blocks each, and the fastest measured candidate was selected.

G. Baselines

All 1,396 feasible configurations were measured once (the exhaustive sweep), and the top 20 re-measured in 3 more blocks, to provide ground truth. Random search [19] was given the same budget as DOE–RSM, repeated 10,000 times by sampling without replacement from the exhaustive measurements; the evaluations-to-match curve uses 2,000 random orderings of the same measurements. For cuBLAS [20], each configuration was benchmarked in the same process, alternating 30-launch blocks of each (10 per process, 3 randomised processes: 900 launches per side), with cuBLAS in pedantic FP32 mode and TF32 disabled.

H. Implementation

Each configuration is compiled by `nvcc` into its own benchmark binary (`cuda/bench.cu` with `cuda/gemm_kernel.cuh`), cached by a hash of the sources and flags. A Python driver (`pipeline.py`) generates each design, randomises the run order, locks the clocks, monitors the GPU through NVML and records every launch; it runs phase by phase (stability, screening, CCD, validation, exhaustive, recheck, comparison). `analysis.py` fits the

models with statsmodels [21] and runs the optimisation with SciPy.

IV. Results and Discussion

A. Screening (RQ1)

The 32-run fractional factorial found all six factors significant at 4096^3 ($R^2 = 0.959$). The unroll factor dominates, with 54% of the variance including its interactions. Raising UNROLL from 1 to 8 adds 7.9 TFLOPS on average and TM from 4 to 8 adds 4.9; the strongest interaction, BN × UNROLL, is negative (−3.1) (Fig. 1). The retention rule kept all six factors; the weakest, BM, still had $p = 0.0026$.

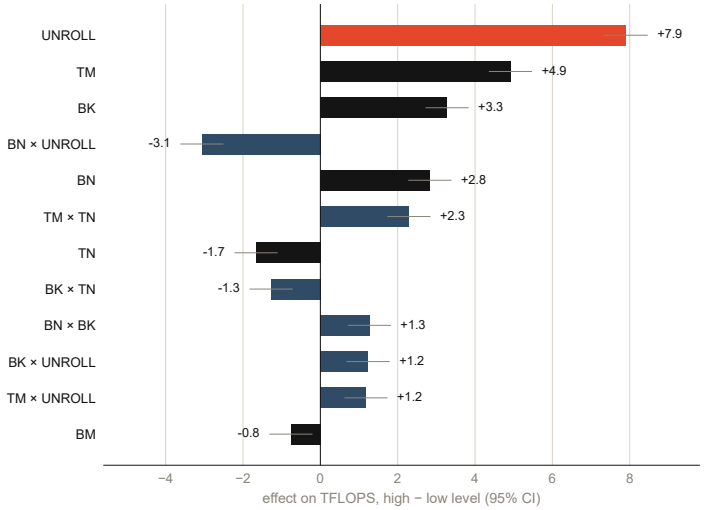


Fig. 1. Screening effects at 4096^3 with 95% confidence intervals: six main effects and the six largest 2-factor interactions. All are significant.

Diagnostics: Randomising the run order removed the autocorrelation that a fixed order produced in a pilot run: the Durbin–Watson statistic on run order is 1.86, against 0.60 in the pilot. There is no drift between blocks ($p = 1.0$) and replicate noise is small (median coefficient of variation 0.037%). Residual variance is not constant (Breusch–Pagan $p < 0.001$), but refitting on log latency gives the same significant factors at all three shapes. Because measurement noise is so small, the residuals are mostly real 3-factor interactions the model omits; that inflates the error estimate, so the tests are conservative.

Effect of matrix shape: Factor rankings at 4096^3 and 2048^3 agree exactly (Kendall $\tau = 1.0$); at 1024^3 they do not ($\tau = -0.07$): BM rises from 1.9% to 23.6% of the variance and TM falls from 22.7% to 3.2% (Fig. 2). A likely cause is GPU fill: at 1024^3 a 128 × 128 tile yields only 64 thread blocks for the H200's 132 SMs, so tile size decides how much of the GPU is used. Tuned settings should not be carried from large to small matrices.

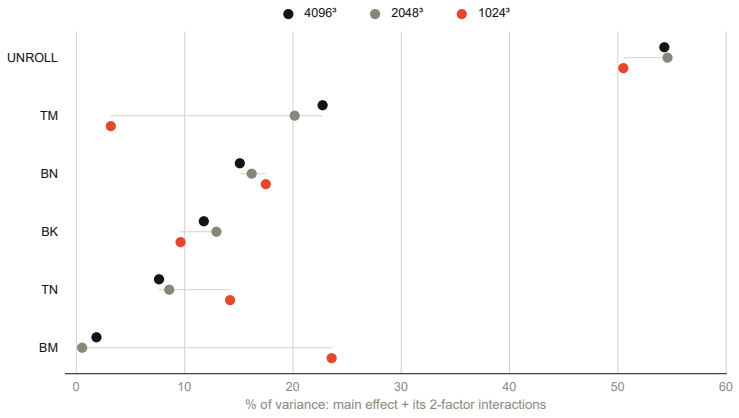


Fig. 2. Share of variance per factor (main effect plus its 2-factor interactions) at three matrix sizes.

B. Response Surface (RQ2)

A second-order surface explains 95.6% of the variance but fails the pre-registered adequacy test: its lack of fit is 7.8% of the model mean square against a 5% limit (Table II). The answer to RQ2 is "close, but not adequate". The lack-of-fit F-test is overwhelming ($p < 10^{-150}$) because replicate noise is only about 0.05%, so even small systematic errors are detectable. 80% of the misfit lies inside the factorial core rather than at the axial extremes, and canonical analysis shows the fitted quadratic is a saddle with no interior maximum.

TABLE II
RESPONSE-SURFACE MODELS

MODEL	PARAMS	R^2	PRED. R^2	LACK OF FIT	GATE (≤ 5%)	HELD-OUT ERROR, MEDIAN / MAX
Second order, pre-registered	30	0.956	0.932	7.8%	fails	10.6% / 46%
Second order, log latency (fallback)	30	0.961	0.940	7.0%	fails	11.4% / 38%
Second order + 3-factor terms	40	0.982	0.970	10.1%	fails	32.6% / 412%

Params include 2 block effects. Lack of fit is the lack-of-fit mean square as a percentage of the model mean square. Held-out error leaves out one configuration (all its replicates) at a time.

The 3-factor investigation: After the gate failed, the author chose to report the failure and then test the obvious remedy: adding the 10 estimable 3-factor interaction pairs (the half-fraction core aliases each triple with its complement). Six are significant, the strongest being BM·BN·BK = TM·TN·UNROLL ($p = 1.4\times10^{-13}$), and together they explain 59% of the quadratic's residual. The augmented model still fails the gate and is overfitted: 40 parameters on 45 distinct configurations make its error on a held-out configuration three times worse. The ordinary predicted R^2 hides this, because it leaves out one measurement at a time while the other replicates of that configuration stay in the training set.

Part of the misfit is not polynomial at all. For (BM, BN, BK, TM, TN) = (64, 64, 32, 8, 4), throughput is non-monotonic in the unroll factor (Fig. 3). A smooth surface cannot follow a compiler code-generation jump like that.

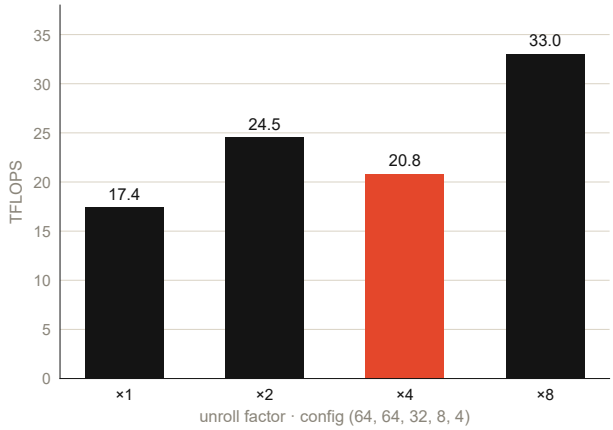


Fig. 3. Throughput against unroll factor for configuration (64, 64, 32, 8, 4): unrolling by 4 is slower than by 2.

Checked against the exhaustive sweep (Fig. 4), both models are usable inside the region they were fitted on (median error 5.5% for the quadratic, 5.0% augmented) and unreliable outside it (21.8% and 22.7%), sometimes predicting negative throughput. The model's best configuration outside the fitted region was predicted at 37.1 TFLOPS and measured at 30.5. This is why optimisation was restricted to the fitted region.

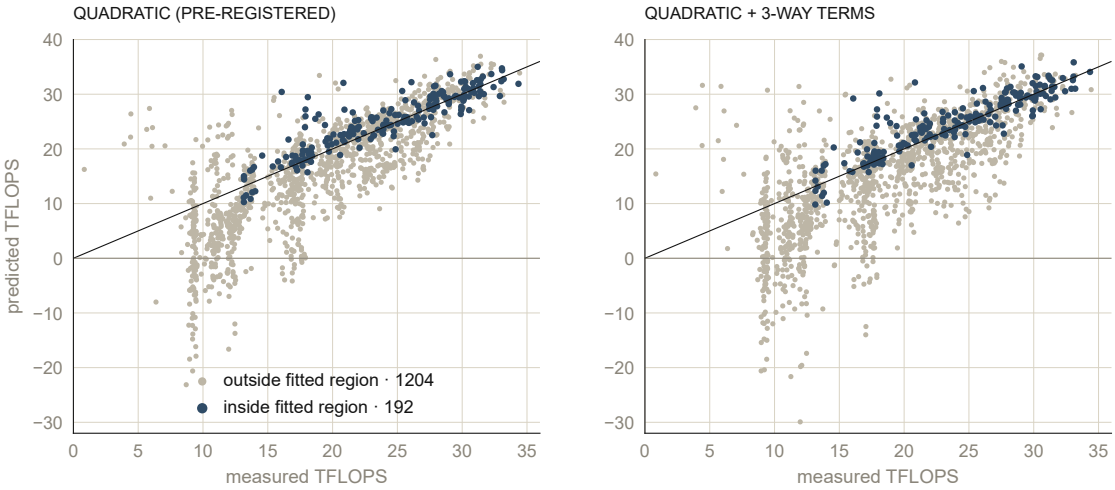


Fig. 4. Predicted against measured throughput for all 1,396 feasible configurations, for both models. Blue: inside the jointly tested region; grey: outside.

The residuals of the pre-registered model (Fig. 5) show no run-order structure (Durbin–Watson 2.15) and no influential point (largest Cook's distance 0.049). They are, however, not normal (Shapiro–Wilk $p = 0.006$) and their variance is not constant (Breusch–Pagan $p < 10^{-8}$); the log-latency refit removes the first problem ($p = 0.08$) but not the second. The failure is systematic misfit, not noise or outliers. Fig. 6 shows the fitted surface for three of the largest 2-factor interactions in Fig. 1.

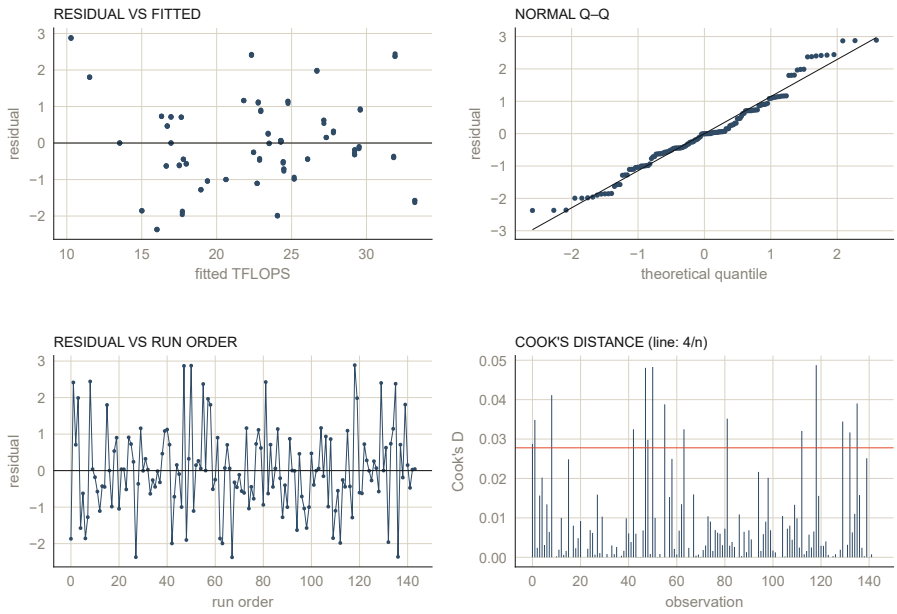


Fig. 5. Residual diagnostics for the pre-registered second-order model.

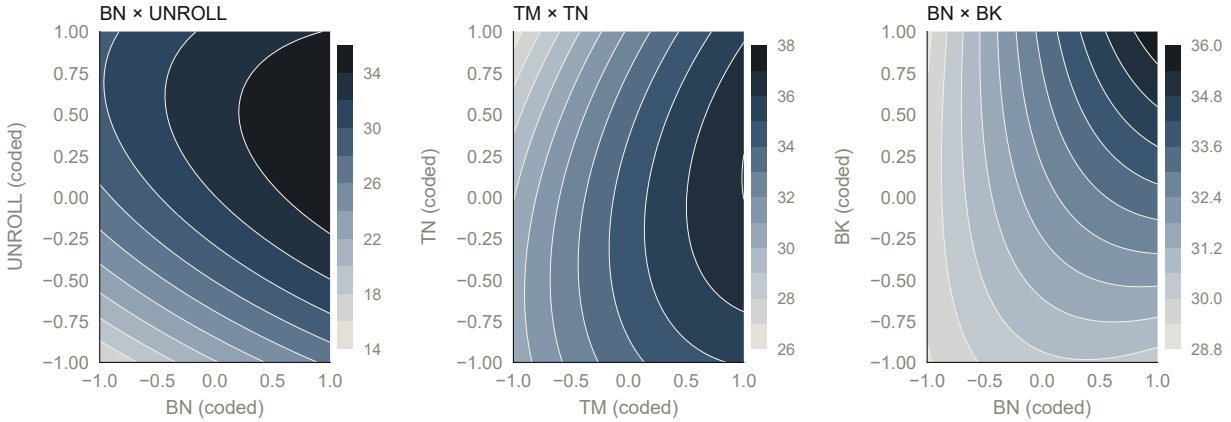


Fig. 6. Fitted surface for three of the largest 2-factor interactions (augmented model; it fails the adequacy gate and is shown for interpretation only). Other factors are held at the model's best configuration.

C. Optimisation and Validation

Measured validation, not the model, picked the final configuration: (128, 128, 32, 8, 8, 8) at 34.32 TFLOPS, which neither model ranked first. The augmented model's top 5, the quadratic's top 3 and the SQP neighbourhood gave 15 distinct candidates (Table III); all 30 SQP starts converged.

TABLE III
VALIDATION CANDIDATES

CONFIGURATION	MEASURED TFLOPS (SD)	AUGMENTED PRED.	QUADRATIC PRED.	RANK: MEAS.	AUG.	QUAD.
(128, 128, 32, 8, 8, 8), selected	34.32 (0.04)	34.07	31.90	1	3	11
(128, 128, 32, 8, 8, 4)	33.08 (0.01)	35.83	34.39	2	1	3
(64, 128, 32, 8, 8, 4)	33.05 (0.03)	33.23	34.73	3	6	2
(64, 64, 32, 8, 4, 8)	33.02 (0.02)	32.57	32.37	4	11	8
(64, 128, 16, 8, 8, 4)	32.21 (0.02)	32.95	33.69	5	9	4
(64, 128, 32, 8, 4, 4)	31.17 (0.03)	33.07	35.01	8	8	1
(64, 64, 32, 8, 4, 4)	20.84 (0.01)	32.15	32.07	15	13	10

The five fastest, each model's first choice and the largest miss, of 15 candidates measured in 5 blocks each. Configurations are (BM, BN, BK, TM, TN, UNROLL).

Within this narrow top band both models rank poorly (Spearman correlation with measured throughput 0.25 augmented, 0.37 quadratic; mean prediction error 9.0% and 8.5%). Trusting a model's own first choice would have cost

3.6% (augmented) or 9.2% (quadratic) of throughput; benchmarking a candidate set instead of a single rounded optimum made the difference.

Against the exhaustive optimum: The true best configuration, (64, 128, 64, 8, 8, 8) at 34.40 TFLOPS, has BK = 64, outside the region the models were allowed to search, so the restriction chosen to avoid extrapolation also put it out of reach. In the re-measurement of the top 20, the selected configuration is second, 0.17% slower, a borderline-significant difference (Welch p = 0.042); the third is 33.22 TFLOPS.

D. Sample Efficiency (RQ3, RQ4)

DOE–RSM consumed 59 unique configurations: 32 in screening, 13 new ones in the CCD and 14 new ones in validation. With these it landed 0.17% from the optimum (Table IV, Fig. 7). Random search with the same budget matched or beat DOE–RSM in 7.8% of 10,000 trials (Fig. 8) and needs a median of 424 evaluations (10th–90th percentile 77–936), about 7× the budget, to reach the same result.

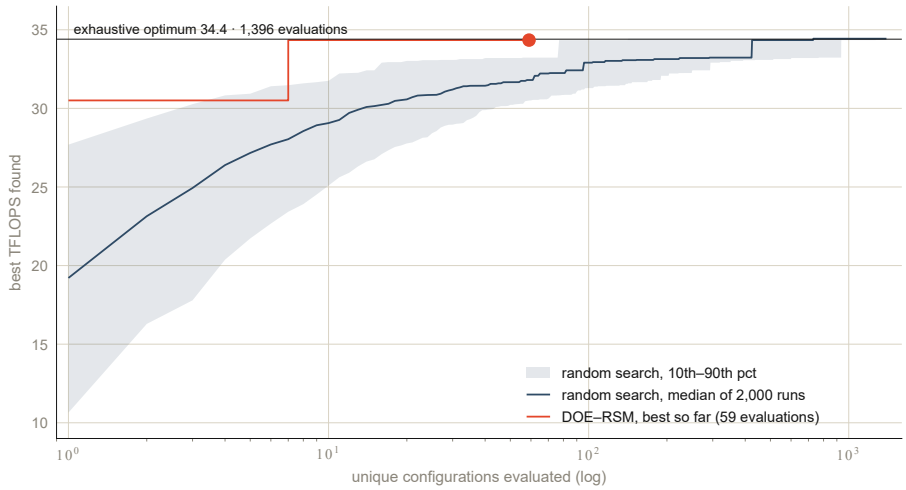


Fig. 7. Best throughput found against configurations evaluated (log scale). DOE–RSM in vermilion; random-search median of 2,000 random orderings as a dark line and its 10th–90th percentile band in grey.

TABLE IV
EQUAL-BUDGET COMPARISON

METHOD	EVALS.	BEST TFLOPS	GAP
DOE–RSM (this work)	59	34.34	0.17%
Random search, same budget: median (10th–90th pct.)	59	32.07 (30.50–33.23)	7.26%
Exhaustive search	1,396	34.40	0%
Untuned default (64, 64, 8, 4, 4, 1)	1	13.15	161%

Gaps are in latency against the exhaustive optimum (3.995 ms). TFLOPS from the exhaustive sweep.

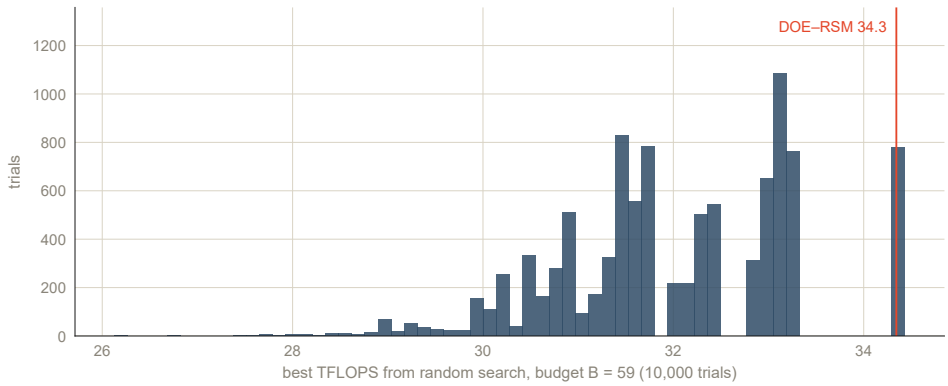


Fig. 8. Distribution of random search's best result at the same 59-evaluation budget, 10,000 trials.

What to credit to which stage: The selected configuration is the all-high corner of the screening design and was first measured at evaluation 7. The response surface and validation confirmed it, by measuring it fastest among 15 model-proposed candidates, but did not discover it. Its presence in the screening design follows from where the screening levels were placed (by engineering judgement, before any

measurement). The efficiency gain belongs to the design as a whole, screening included, not to the fitted surface alone.

The selected configuration appears as 34.34 TFLOPS in Table IV (exhaustive sweep), 34.32 in Table III (validation) and 34.35 in Table V (cuBLAS comparison). The differences, under 0.1%, come from separate measurement sessions.

E. Comparison with cuBLAS (RQ5)

The selected kernel attains 72.0% of cuBLAS FP32 throughput and is 2.61× faster than the untuned default (Table V, Fig. 9). cuBLAS's default mode gave identical times to pedantic FP32 mode, and its output matches the custom kernel's bit for bit, so the comparison is FP32 against FP32.

TABLE V
INTERLEAVED COMPARISON WITH CUBLAS, 4096^3, PEDANTIC FP32

CONFIGURATION	LATENCY (MS)	CUSTOM TFLOPS	CUBLAS TFLOPS	SHARE
DOE–RSM selection (128, 128, 32, 8, 8, 8)	4.001	34.35	47.71	72.0%
Exhaustive best (64, 128, 64, 8, 8, 8)	3.993	34.42	47.68	72.2%
Untuned default (64, 64, 8, 4, 4, 1)	10.447	13.16	47.69	27.6%

Median of 900 interleaved launches per side.

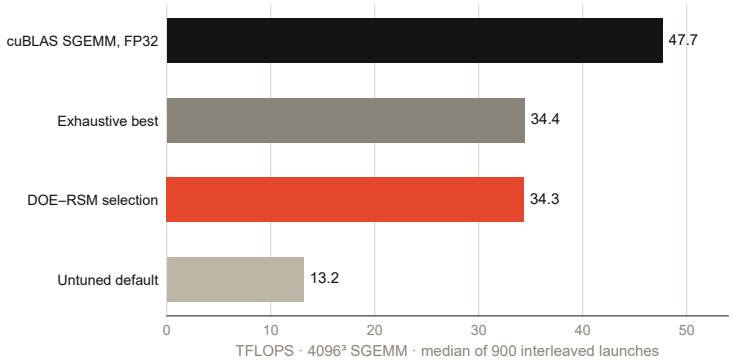


Fig. 9. Throughput of the untuned default, the DOE–RSM selection, the exhaustive best and cuBLAS.

The remaining 28% is a ceiling of the kernel design, not of the tuning: even the best of all 1,396 configurations stops at 72.2%. cuBLAS uses techniques this kernel does not have (vectorised 128-bit loads, asynchronous double-buffered copies, warp-level tiling); adding them would change the kernel family and lies outside this study.

F. Register Spilling

Register spilling is rare in this kernel family and does not drive its non-linear behaviour: 28 of 1,396 configurations spill, and they are not systematically slower (Fig. 10). The compiler's resource report was recorded for every configuration. Register use ranges from 20 to 255 per thread. Spilling configurations have a median of 21.6 TFLOPS against 19.0 for the rest; most sit at 64 registers, the per-thread limit a 1,024-thread block imposes. Throughput rises with registers per thread (Spearman 0.47) and falls with occupancy (−0.45): larger register tiles reuse each loaded value more, which outweighs having fewer threads resident. Register spilling was expected to be a source of non-linearity; in this kernel the non-linearity comes instead from register-tile reuse, unroll-dependent code generation and, at small shapes, GPU fill.

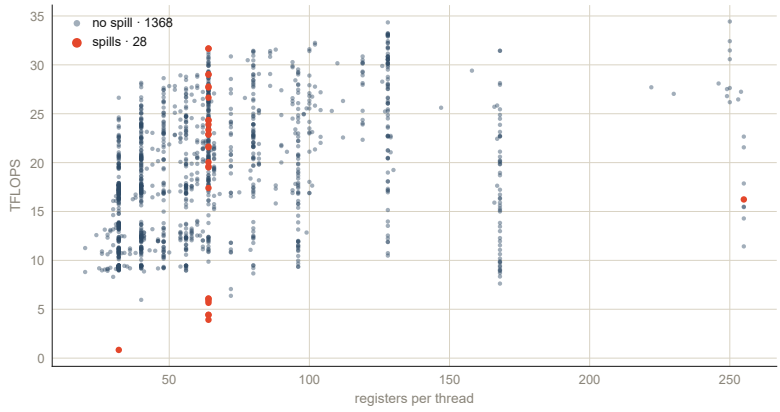


Fig. 10. Registers per thread against throughput for all 1,396 feasible configurations; spilling configurations in vermilion.

G. Deviations and Threats to Validity

One deviation from the pre-registered contract was made after seeing data; everything else ran as written.

Deviation: The contract said not to optimise a surface that fails the adequacy gate. After it failed, the author chose to

report the failure (Section IV-B) and continue with the 3-factor-augmented model as a proposal generator, selecting the final configuration only by measured validation. Both models' recommendations were benchmarked, so the effect of that choice is visible in Table III.

Shared GPU: Contested measurement windows were detected through NVML and repeated (37 repeats, 1 still flagged, kept and marked as the contract requires). NVML samples utilisation periodically and can miss very short bursts; the median over 30 launches limits their influence.

Clock: The requested 1980 MHz lock was held at 1830 MHz by the driver. Every measurement ran at that clock, so all throughputs, cuBLAS included, are about 7.5% below unlocked boost figures; ratios are unaffected.

Scope: One GPU, FP32 on CUDA cores, one kernel family and one primary shape (4096^3). The 1024^3 screen shows factor importance changes with shape, so the selected configuration should not be carried to other shapes without re-tuning.

Power-of-two lattice: Coded design points were snapped to legal levels, so the CCD is not exactly rotatable and its centre runs at (64, 64, 16, 4, 4, 2); models were fitted on the realised levels.

Where the winner came from: The selected configuration is a corner of the screening design, so the result depends partly on where the screening levels were placed, and the true optimum lies outside the region the models were allowed to search.

Ground truth: Random search was scored with the single-block exhaustive measurements; the top 20 were re-measured in 3 more blocks and kept their order.

Correctness reference: At 4096^3 cuBLAS sums in the same order as this kernel, so their outputs are identical; agreement with an independent float64 CPU reference was checked for the default and selected configurations at 512^3 and 1024^3.

Pilot data excluded: An earlier pilot run without clock locking or randomisation shaped this protocol; none of its measurements are used here.

V. Conclusion and Future Scope

The answers to the research questions are as follows. RQ1: yes; the fractional factorial separated all six factors ($R^2 = 0.959$), though their importance shifts at 1024^3. RQ2: close but inadequate; lack of fit was 7.8% against a 5% limit, caused by 3-factor interactions and compiler jumps. RQ3: yes; 59 evaluations against a median of 424 for random search with the same goal. RQ4: 0.17% from the exhaustive optimum, the second-best of 1,396 configurations. RQ5: 72.0% of cuBLAS FP32 throughput and 2.6× the untuned kernel.

The practical lesson is to use the response surface to propose candidates and let measurement decide. Four extensions follow:

1. a full 2^6 factorial core, so 3-factor interactions can be estimated without overfitting;
2. tuning per matrix shape, since importance shifts below 2048^3;
3. a head-to-head comparison with Bayesian optimisation, the method modern autotuners use;
4. kernel features such as vectorised loads and double buffering as categorical factors, to raise the 72% ceiling.

Appendix
Reproducibility

Every number in this report comes from run `final_20261028` and can be regenerated from its raw files (Table VI). The experimental contract (`contract.json`, SHA-256 prefix `857dc5fbcf29`) was frozen before data collection and its hash is stamped on the run. The run is executed with `python pipeline.py <run> <phase>` and analysed with `python analysis.py <run> <phase>`.

Table VI
Data and Code

File	Contents
`results/final_20261028/measurements.csv`	2,001 measurement blocks: configuration, validity, clock, temperature, contention flags, registers, median and IQR latency
`results/final_20261028/raw_launches.csv`	every individual timed launch
`results/final_20261028/decisions/`	each automatic decision: retained factors, CCD design, validation candidates, final selection
`results/final_20261028/analysis/`	ANOVA tables, model coefficients, cross-validation, summaries
`results/feasibility_audit.csv`	all 2,304 candidates with feasibility status and the compiler's verdict
`cuda/gemm_kernel.cuh`, `cuda/bench.cu`	the kernel and its benchmark harness

References

[1] T. Chen *et al.*, "TVM: An automated end-to-end optimizing compiler for deep learning," in *Proc. 13th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, Carlsbad, CA, USA, 2018, pp. 578–594.

[2] L. Zheng *et al.*, "Ansor: Generating high-performance tensor programs for deep learning," in *Proc. 14th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2020, pp. 863–879.

[3] G. E. P. Box and K. B. Wilson, "On the experimental attainment of optimum conditions," *J. Roy. Statist. Soc. Ser. B*, vol. 13, no. 1, pp. 1–45, 1951.

[4] D. C. Montgomery, *Design and Analysis of Experiments*, 9th ed. Hoboken, NJ, USA: Wiley, 2017.

[5] J. Ansel *et al.*, "OpenTuner: An extensible framework for program autotuning," in *Proc. 23rd Int. Conf. Parallel Architectures and Compilation (PACT)*, Edmonton, AB, Canada, 2014, pp. 303–316.

[6] C. Nugteren and V. Codreanu, "CLTune: A generic auto-tuner for OpenCL kernels," in *Proc. IEEE 9th Int. Symp. Embedded Multicore/Many-core Systems-on-Chip (MCSoC)*, Turin, Italy, 2015, pp. 195–202.

[7] B. van Werkhoven, "Kernel Tuner: A search-optimizing GPU code auto-tuner," *Future Gener. Comput. Syst.*, vol. 90, pp. 347–358, 2019.

[8] F.-J. Willemsen, R. van Nieuwpoort, and B. van Werkhoven, "Bayesian optimization for auto-tuning GPU kernels," in *Proc. IEEE/ACM Int.*

Workshop Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS), 2021, pp. 106–117.

[9] V. Volkov and J. W. Demmel, "Benchmarking GPUs to tune dense linear algebra," in *Proc. ACM/IEEE Conf. Supercomputing (SC)*, Austin, TX, USA, 2008, pp. 1–11.

[10] R. H. Myers, D. C. Montgomery, and C. M. Anderson-Cook, *Response Surface Methodology: Process and Product Optimization Using Designed Experiments*, 4th ed. Hoboken, NJ, USA: Wiley, 2016.

[11] D. B. Kirk and W. W. Hwu, *Programming Massively Parallel Processors: A Hands-on Approach*, 3rd ed. Cambridge, MA, USA: Morgan Kaufmann, 2016.

[12] NVIDIA Corporation, *CUDA C++ Programming Guide*, Release 13.0, 2025. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/

[13] J. Durbin and G. S. Watson, "Testing for serial correlation in least squares regression. II," *Biometrika*, vol. 38, no. 1/2, pp. 159–177, 1951.

[14] T. S. Breusch and A. R. Pagan, "A simple test for heteroscedasticity and random coefficient variation," *Econometrica*, vol. 47, no. 5, pp. 1287–1294, 1979.

[15] S. S. Shapiro and M. B. Wilk, "An analysis of variance test for normality (complete samples)," *Biometrika*, vol. 52, no. 3/4, pp. 591–611, 1965.

[16] R. D. Cook, "Detection of influential observation in linear regression," *Technometrics*, vol. 19, no. 1, pp. 15–18, 1977.

[17] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed. New York, NY, USA: Springer, 2006.

[18] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[19] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *J. Mach. Learn. Res.*, vol. 13, pp. 281–305, 2012.

[20] NVIDIA Corporation, *cuBLAS Library User Guide*, CUDA Toolkit 13.0, 2025. [Online]. Available: https://docs.nvidia.com/cuda/cublas/

[21] S. Seabold and J. Perktold, "Statsmodels: Econometric and statistical modeling with Python," in *Proc. 9th Python in Science Conf. (SciPy)*, Austin, TX, USA, 2010, pp. 92–96.